Working with Models:

- In web application development, we must interact with a database to store data and retrieve stored data.
- Django provides strong built-in support for database operations.
- Django comes with a built-in database called SQLite3.
- SQLite3 is enough for small to medium applications.
- Django also supports other databases like Oracle and MySQL.
- A Model is the main source of information about your data. It defines the fields and behavior of the data you want to store.
- Generally, one model represents one database table
- Each model is a Python class that inherits from django.db.models.Model.
- Each variable (attribute) inside the model represents a column in the database table.
- Django automatically creates an API to interact with the database. So, we do not need to write SQL queries manually.
- API is ORM **(Object Relational Mapper)** - ORM allows us to talk to database using Python code instead of SQL.

  Ex: SQL - SELECT * FROM student;

  ORM - Student.objects.all()

Ex:

Create a Django application that performs full CRUD operations (Create, Read, Update, Delete) on database records and displays the data as a list. (Save objects → Create, Fetch data → Read, Update records → Update, Delete records → Delete, Show as list → Display)

Step 1: Create Project and Application

```
python -m django startproject ModelProject
python .\manage.py startapp ModelApp
```

Step 2: Register the application in setting file.

```
INSTALLED_APPS = [
    ....
    'ModelApp',
]
```

Step 3: Django already provides the following congiguration. No need to change for basic project.

Suresh Yadlapati

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

Step 4: Create Model (Table)

```
from django.db import models

class Student(models.Model):
        name = models.CharField(max_length=100)
        age = models.IntegerField()
        course = models.CharField(max_length=100)
```

- We are creating a model. (Model == Table and fields = = columns)
  - It must inherit from models.Model.
  - name = models.CharField(max_length=100)
  - This creates a column which is of text type.
  - Text type - max_length is compulsory.
  - age = models.IntegerField()
  - This creates number column.

- Common Model Field Types (Tell Students)
  - CharField → text
  - TextField → long text
  - IntegerField → numbers
  - FloatField → decimal
  - BooleanField → True/False
  - DateField → date
  - EmailField → email
  - ImageField → images

Step 5: Create Migration File

At project level issue the command python manage.py makemigrations

What it does

- Django checks your models.py
- It finds new models or changes
- It creates a migration file

Example migration file:

ModelApp/migrations/0001_initial.py

Suresh Yadlapati

This file contains instructions like:

- Create table
- Add column
- Modify column
- Delete column

So makemigrations does NOT change the database yet.

It only prepares instructions.

Step 6: Issue migrate – Apply Changes to Database and to create Table.

python manage.py migrate

What it does

- Reads the migration files
- Executes them on the database
- Creates or updates tables

Example:

If you create this model:

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
```

After running migrate, Django creates a table like:

ModelApp_student

with columns: id | name | age

Command	Purpose	Example
makemigrations	Create the instructions	Create building plan
migrate	Apply instructions to database	Construct the building

Step 7: Register Model in Admin

```
from django.contrib import admin
# Register your models here.admin
from .models import Student
admin.site.register(Student)
```

Suresh Yadlapati

Step 8: Create Superuser (To work from admin panel)

Username (leave blank to use 'admin'): admin

Email address: sureshyadlapati@gmail.com

Password: admin@123

Password (again):admin@123

The password is too similar to the username.

Bypass password validation and create user anyway? [y/N]: y

Superuser created successfully.

Step 9: Run the Server and login as super user and observe.

CRUD operations in Django can be performed using Forms, the Admin Panel, and the Django Shell**.** I want to perform the operations (commands) through Shell.

Step 1: Using Shell: python manage.py shell - To exit from shell type exit()

Step 2: To create record in table

```
s1 = Student(name="Suresh", age=19, course="B Tech")
s1.save()
```

- Data inserted into database
- ORM converts into SQL automatically

Step 3: READ (Fetch Data)

```
>>> students = Student.objects.all()
>>> for s in students:
...     print(s.name, s.age, s.course)
Suresh 19 B Tech
```

Other ways: `Student.objects.get(id=1)`

`Student.objects.filter(name="Ramesh")`

Step 4: UPDATE (Modify Data)

- Get object first:

```
student = Student.objects.get(id=1)
```

- Change the value

Suresh Yadlapati

```
student.age = 25
student.save()
```

Ex:

```
>>> s= Student.objects.get(id=1)
>>> s.age
19
>>> s.age=20
>>> s.save()
```

Step 5: Delete

- Get object: student = Student.objects.get(id=1)
- Delete: student.delete()
- Record will be removed from database.

Note: to see all the records in a table use this command:

```
>>> Student.objects.all().values()
<QuerySet [{'id': 1, 'name': 'Suresh', 'age': 20, 'course': 'B Tech'}]>
```

<u>Displaying using Template:</u>

- After creating Models, with the fields and data we want in them, it is time to display the data in a web page.

Creation of Template: Create a templates folder in main project. Now create html page.

Ex: Home.html

And also add the following statement in **settings.py** file

'DIRS': ['F:\S1-S2\ModelProject\\templates'],

views. py

```
from django.shortcuts import render
from ModelApp.models import Student
# Create your views here.
def show_students(request):
    students = Student.objects.all()
    return render(request, 'home.html', {'students': students})
```

Home.html

```
<html>
  <head>
    <title>Student List</title>
```

```
    </head>
    <body>
        <h2>Student List</h2>
        <table border="1">
        <tr>
            <th>Name</th>
            <th>Age</th>
            <th>Course</th>
        </tr>
        {% for s in students %}
        <tr>
            <td>{{ s.name }}</td>
            <td>{{ s.age }}</td>
            <td>{{ s.course }}</td>
        </tr>
        {% endfor %}
    </table>
  </body>
  </html>
```

urls.py

```
from ModelApp import views
urlpatterns = [
        path('admin/', admin.site.urls),
       path('', views.show_students),
]
```

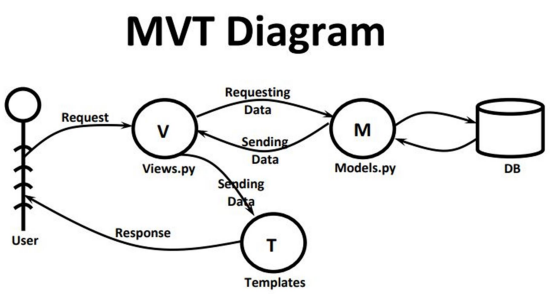
Run the Server:

(PVPSIT) PS F:\S1-S2\ModelProject> python .\manage.py runserver

Output:

Student List

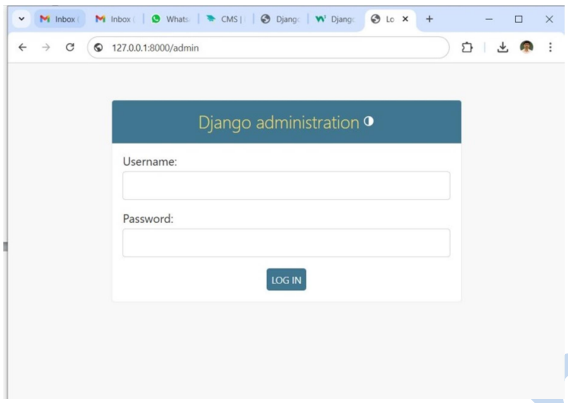
Name	Age	Course
Suresh	20	B Tech
Pavan	40	M Tech
Ravi	20	BCA
Sita	21	BSc
Rahul	22	BTech



Suresh Yadlapati

Django Admin – Creation of Superuser:

- Django Admin is a really great tool in Django, it is actually a CRUD* user interface of all our models!. CRUD stands for Create Read Update Delete.



Create User:

- To be able to log into the admin application, we need to create a user.
- This is done by typing this command in the command view:

  (DJANGOWS_YS) PS F:\DJANGOWS_YS\WorkingWithModels> python manage.py createsuperuser

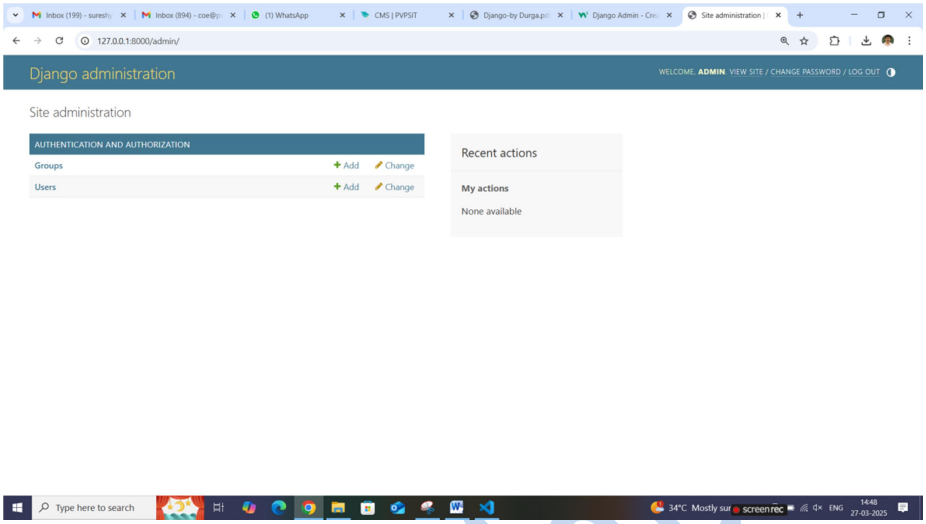
Which will give this prompt:

```
(DJANGOWS_YS) PS F:\DJANGOWS_YS\WorkingWithModels> python manage.py createsuperuser
Username (leave blank to use 'admin'):
Email address: suresh@pvpsiddhartha.ac.in
Password:
Password (again):
The password is too similar to the email address.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
(DJANGOWS_YS) PS F:\DJANGOWS_YS\WorkingWithModels>
```

Password is: admin@123 and username is admin

In the browser window, type 127.0.0.1:8000/admin/ in the address bar. And fill in the form with the correct username and password.

Which should result in this user interface:



Missing Model: The Members model is missing, as it should be, you have to tell Django which models that should be visible in the admin interface.

Include Member(Model class) in the Admin Interface:

- To include the Member model in the admin interface, we have to tell Django that this model should be visible in the admin interface.
- This is done in a file called admin.py, and is located in your app's folder, which in our case is the ModelApp folder. Open it, and it should look like this:

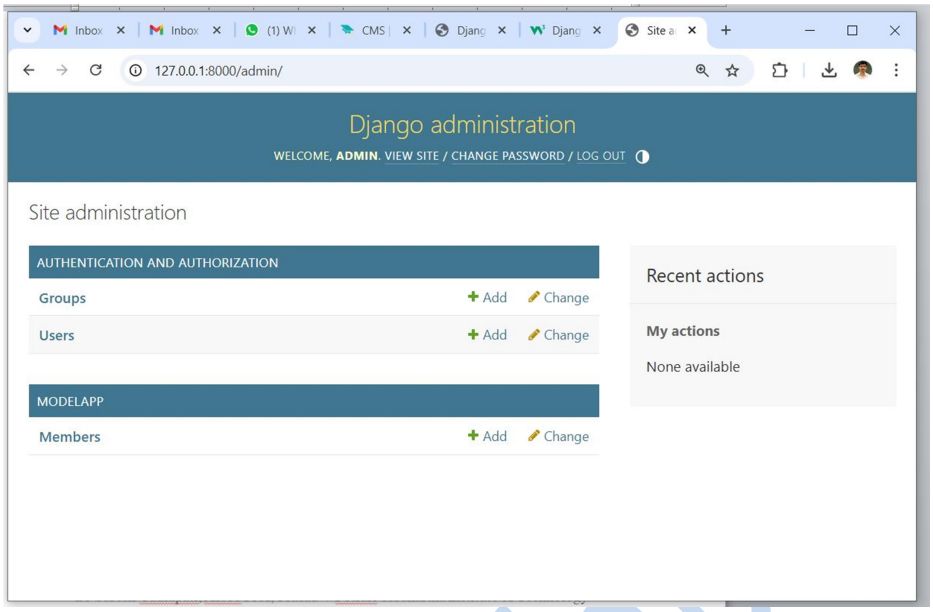
```
from django.contrib import admin

# Register your models here.
```

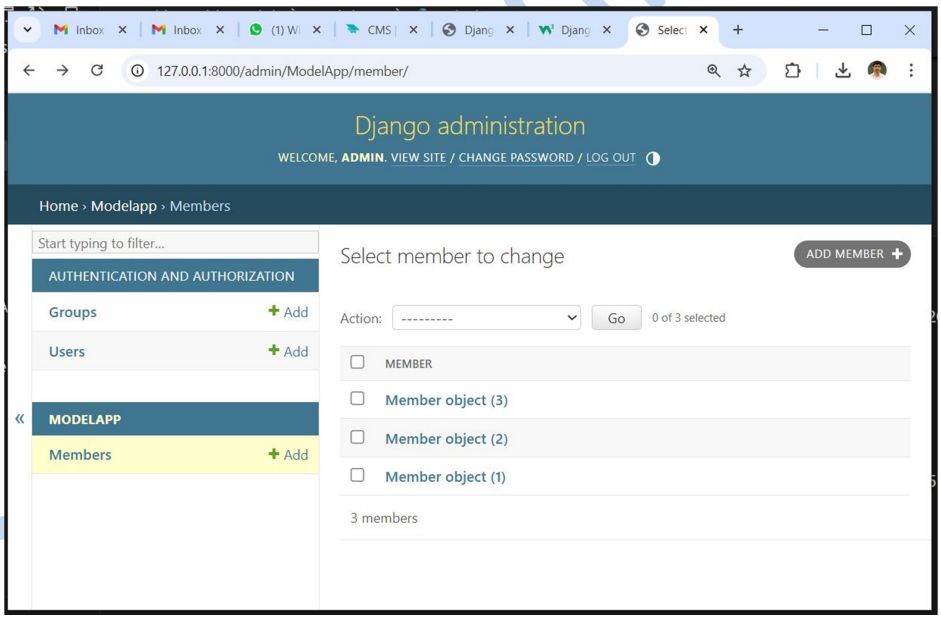
- Insert a couple of lines here to make the Member model visible in the admin page:

```
from .models import Member

# Register your models here.
admin.site.register(Member)
```



- Click Members and see the records we inserted earlier:



Change Display:

- In the list in the screenshot above, we see "Member object (1)", "Member object (2)" etc. which might not be the data you wanted to be displayed in the list.
- It would be better to display "firstname" and "lastname" instead.

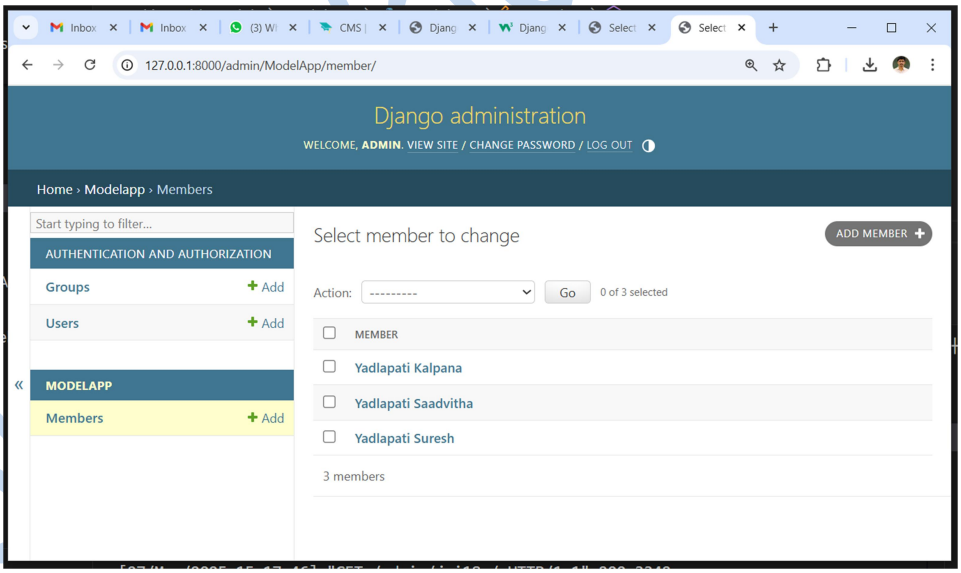
- This can easily be done by changing some settings in the **models.py** and/or the **admin.py** files.
- To change this to a more reader-friendly format, we have two choices:
    1. Change the string representation function, __str__() of the Member Model
    2. Set the list_display property of the Member Model

1. Change the String Representation Function: To change the string representation, we have to define the **__str__()** function of the Member Model in models.py, like this:

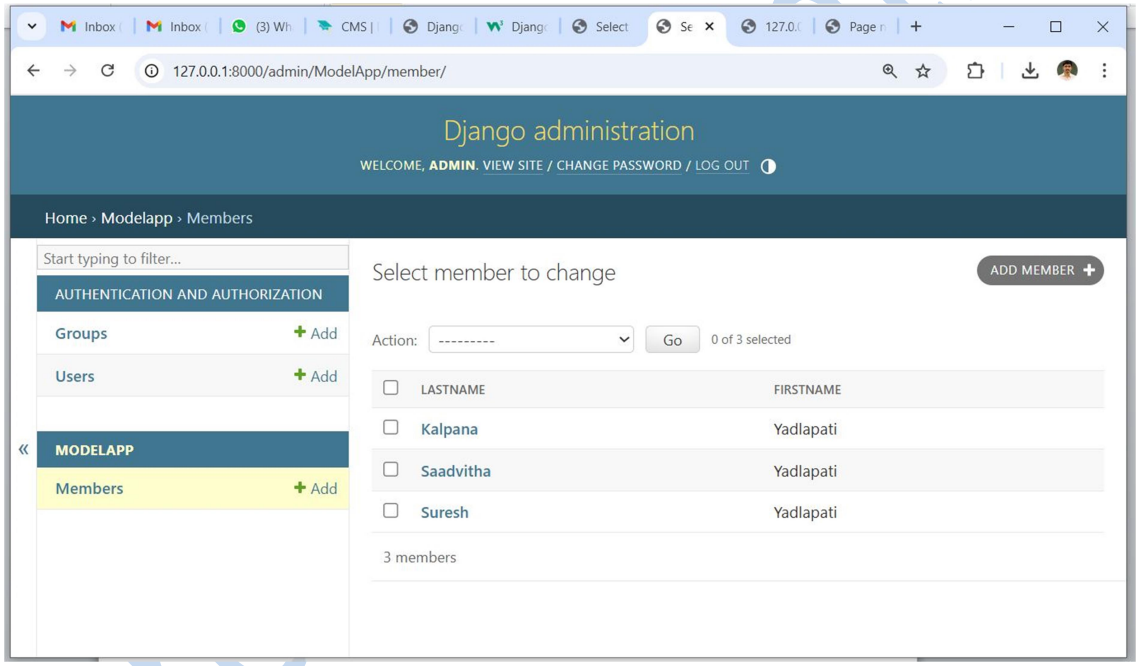
```
from django.db import models
# Create your models here.

class Member(models.Model):
  firstname = models.CharField(max_length=255)
  lastname = models.CharField(max_length=255)

  def __str__(self):
    return f"{self.firstname} {self.lastname}"
```



2. Set list_display:

- We can control the fields to display by specifying them in a list_display property in the **admin.py** file. (last name first and first name last or vice versa)
- First create a MemberAdmin() class and specify the list_display tuple, like this:

```
from django.contrib import admin
from .models import Member
# Register your models here.

class MemberAdmin(admin.ModelAdmin):
  list_display = [ "lastname", "firstname",]


admin.site.register(Member, MemberAdmin)
```